Cognitive Kernel Networks (CKN)

Architectural Geometry for Privileged Reasoning in Prefix-Conditioned Transformers

John P. Alioto

December 2025

Protocol v0.2.0

Abstract

Prefix-conditioned transformers compute within a learned high-dimensional vector field. Previous work on Cognitive Tensor Networks (CTN) showed that structured prefix geometry can shape inference trajectories and reduce drift, revealing an architectural limitation: transformers lack a privilege boundary separating user-level input from system-level reasoning. Any structured prefix—benign or adversarial—can distort the model's internal trajectory.

This paper introduces Cognitive Kernel Networks (CKN), the architecture-level analogue of CTN. Whereas CTN constrains reasoning through structured prefix geometry (client space), CKN defines persistent architectural kernels (server space): privileged reasoning subspaces, frozen invariants, and high-dimensional attractors that external input cannot span.

We hypothesize that CTN and CKN are two expressions of a single underlying control problem: shaping trajectories in a learned vector field under distinct privilege levels. CTN influences initial conditions; CKN influences the topology of the vector field itself. Together they provide a unified, full-stack geometric framework for robust transformer reasoning.

1 Introduction

Large Language Models implement a deterministic dynamical system:

$$h_{t+1} = F_\Theta(h_t, x_t), \tag{1}$$

where h_t is the hidden state, x_t is the input token embedding, and Θ are the learned parameters.

CTN reframed prompting as geometric steering: the prefix acts as a constraint manifold that biases the inference trajectory into a structured subspace. Experiments show that CTN reduces drift and stabilizes reasoning without modifying Θ.

However, CTN exposes a deeper architectural issue: transformers treat all prefixes as equally privileged. System instructions, user queries, safety metadata, and adversarial fragments share a single representational manifold. There is no architectural equivalent of a kernel boundary. This permits prefix perturbations to redirect internal computation, producing unstable manifold transitions and drift-collapse.

This motivates **Cognitive Kernel Networks (CKN)**, defined as architectural kernels that construct privileged reasoning geometry which external tokens cannot significantly perturb.

CKN is not a replacement for CTN. It is the server-side analogue of the same theory:

- **CTN:** prefix geometry (client space) shapes initial inference.
- **CKN:** architectural geometry (kernel space) shapes manifold topology.

We propose that CTN and CKN formalize a single geometric control problem at distinct privilege levels.

2 Background: CTN and Prefix Geometry

CTN represents reasoning as a weighted tensor decomposition:

$$\vec{C}_{net} = \sum_i w_i \vec{v}_i, \tag{2}$$

combined with a decoder manifold optimizing density and structural coherence.

CTN provides:

- deterministic manifold shaping,
- syntax-level invariants,
- reduced drift,
- improved long-context stability.

But CTN also reveals a structural limitation: prefix geometry alone cannot distinguish trusted reasoning from untrusted input. Transformers ingest the entire prefix as a unified control surface, which is insufficient for systems operating over open or adversarial inputs.

The architectural boundary must come from the model itself.

3 Problem Statement: No Privilege Separation

Let $\mathcal{H}$ denote the transformer's hidden state space. Define:

- $U \subset \mathcal{H}$: user-span subspace (input embeddings),
- $R \subset \mathcal{H}$: reasoning subspace (internal computation).

In current architectures:

$$U \approx R, \tag{3}$$

meaning user-level content and system-level computation are entangled within the same manifold.

Consequences include:

- jailbreak susceptibility,
- manifold drift under long prompts,
- prefix-induced topology shifts,
- unstable multi-agent behavior,
- context poisoning.

CTN shapes the prefix; CKN is required to shape the architecture.

4 Introducing Cognitive Kernel Networks (CKN)

We define the architectural kernel:

$$\mathbb{K} = \{R, I, \Pi, \Lambda\}, \tag{4}$$

where:

- R: privileged reasoning subspace,
- I: architectural invariants (frozen or structure-preserving directions),
- Π: routing and attention policies isolating R from U,
- Λ: gain parameters enforcing geometric dominance.

CKN modifies conceptual architecture such that:

1. external tokens operate primarily in U, a low-dimensional subspace;
2. internal reasoning occurs in R, a high-dimensional privileged subspace;
3. architectural dominance holds:
$$\|\Delta h_R\| \gg \|\Delta h_U\|. \tag{5}$$

User-span perturbations cannot significantly affect internal reasoning dynamics.

CKN offers:

- structural stability,
- predictable inference trajectories,
- resistance to prefix-induced drift,
- auditable invariants,
- builder-defined cognitive policy.

5 Unified Hypothesis: CTN and CKN in Different Coordinates

We hypothesize that CTN and CKN constitute two interfaces to the same underlying control problem: *shaping trajectories within a learned vector field under different privilege constraints.*

Let:
$$h_{t+1} = F_\Theta(h_t, x_t, \mathbb{T}, \mathbb{K}), \tag{6}$$
where $\mathbb{T}$ is the CTN kernel and $\mathbb{K}$ the CKN kernel.

Then:

- CTN constrains initial conditions (session-level geometry),
- CKN constrains manifold topology (architectural geometry).

Robust transformer cognition requires both.

6 Formal Model

6.1 Subspace Decomposition

We decompose the state space as:
$$\mathcal{H} = U \oplus R \oplus S, \tag{7}$$
where S denotes slack or auxiliary capacity.

6.2 Privilege Separation

A valid architecture satisfies:
$$\dim(R) \gg \dim(U), \qquad \langle R, U \rangle \approx 0. \tag{8}$$

6.3 Dominance Criterion

Let the perturbation decompose as:

$$\Delta h = \Delta h_R + \Delta h_U. \tag{9}$$

CKN requires:

$$\|\Delta h_R\| \gg \|\Delta h_U\|. \tag{10}$$

6.4 Interaction with CTN

Unified inference requires:

$$h_t \in \mathcal{M}_{CTN} \cap \mathcal{M}_{CKN}. \tag{11}$$

7 Architectural Considerations

CKN is a conceptual framework, not an architectural prescription. Possible implementation choices include:

7.1 Embedding Partition

$$E = [E_U \mid E_R] \tag{12}$$

7.2 Privileged Attention Heads

$$\alpha_R = \text{softmax}(Q_R K_R^\top) \tag{13}$$

7.3 Routing Policies

$$\text{route}(x) = \begin{cases} \text{Expert}_R & x \in R, \\ \text{Expert}_U & x \in U. \end{cases} \tag{14}$$

7.4 Gain Control

$$W_R \leftarrow \lambda W_R, \quad \lambda \gg 1. \tag{15}$$

These are examples of implementations compatible with the CKN framework; they are not required by the theory.

8 Interoperation with CTN

CTN provides session-level geometric constraints (prefix manifold). CKN provides persistent architectural constraints (kernel manifold).

Inference is governed jointly by:

$$(\mathbb{T}, \mathbb{K}) \text{ over } \mathcal{H}. \tag{16}$$

CTN governs how trajectories begin; CKN governs where they are allowed to evolve.

9 Hypothesized Benefits

We hypothesize that CKN yields:

- resistance to prefix-induced manifold drift,
- reduced jailbreak susceptibility,
- long-context stability,
- improved multi-agent coherence,
- explicit cognitive provenance,
- predictable inference behavior.

CTN + CKN provides full-stack cognitive geometry.

10 Scope of Claims

CKN is a theoretical framework for introducing privilege separation into transformer reasoning. Our claims concern the conceptual decomposition of hidden state geometry and its implications for inference stability.

What CKN Claims

- **Geometric Framing.** CTN and CKN are two control surfaces on the same learned vector field.
- **Manifold Decomposition.** Separating $\mathcal{H}$ into user-span (U) and privileged reasoning (R) provides a meaningful structural abstraction.
- **Privilege Separation.** Transformers lack a privileged reasoning basis; CKN provides a framework for defining one.
- **Dominance Requirements.** A stable privileged subspace must satisfy $\|\Delta h_R\| \gg \|\Delta h_U\|$.
- **Unified Control Problem.** CTN governs initial conditions; CKN governs manifold topology.

What CKN Does Not Claim

- **No Weight Modification.** CKN does not modify or require modification of model parameters.
- **No Guaranteed Empirical Gains.** Performance, safety, or robustness improvements are empirical questions.
- **No New Capabilities.** CKN does not endow new reasoning abilities.
- **No Architectural Prescription.** CKN describes a structural abstraction; implementation is optional and variable.
- **No Alignment Guarantee.** CKN concerns geometric stability, not normative goals.

11 Conclusion

CKN introduces architectural privilege separation into transformer reasoning. Together with CTN, it forms a unified theory: transformers compute inside a learned vector field, and both prefix geometry and architectural geometry are control surfaces over that field.

The traditional separation between "prompt engineering" and "model design" is artificial; both shape trajectories in the same latent space. CKN-enabled architectures may serve as a foundation for predictable, stable, and builder-governed transformer cognition.

References

1. Alioto, J.P. (2025). *Cognitive Tensor Networks: Deterministic Latent-Space Steering via Syntactic Constraints.* CTN Whitepaper v0.1.1.
2. Zou, A. et al. (2023). *Representation Engineering: A Top-Down Approach to Steering LLMs.*
3. Wang, L. et al. (2025). *Orthogonal Safety via Subspace Decomposition.*
4. Zhang, M. et al. (2024). *Prompting as Optimal Control.*
5. Elhage, N. et al. (2023). *A Mechanistic Understanding of Transformer Circuits.*
6. Vogel, T. (2024). *repeng: Representation Engineering Library.*
7. Lumpenspace (2024). *Palinor: Tools for Latent Space Steering.* GitHub repository.